

Document number: P0358R0
Date: 2016-05-28
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

Fixes for not_fn

1. Introduction

In this paper new wording for `std::not_fn` is proposed, that amongst other improvements, provides support of propagation of value category in case of wrapper invocation.

2. Motivation and Scope

The main motivation for creation of this paper comes from the realisation that the existing wording for `std::not_fn`, that was recently accepted in the C++17, requires implementation to always perform call on the lvalue of the stored callable.

This effectively requires `std::not_fn` to disregard the reference qualification of nested function. For example in case of the following definitions:

```
struct RvalueCallable
{
    bool operator>() &&;
};

struct LvalueCallable
{
    bool operator>() &;
};

auto rval = std::not_fn(RvalueCallable{});
auto lval = std::not_fn(LvalueCallable{});
```

Both the invocation `rval()` and `std::move(rval)()` are ill-formed, as they will lead to call on the `RvalueCallable&`. In addition the call `std::move(lval)()` will be well-formed despite explicit reference qualification of the functor, that should prevent invocation on temporaries.

It is also important to emphasize that introduction of the value category propagation after standardization of the current wording would be a breaking change. Furthermore such breakage may be silent in case when wrapped callable was providing both lvalue and rvalue overloads of call operator - in case of temporary wrappers, the code will start to invoke rvalue overloads.

3. Wording Discussion

After finding above flaw in the current wording, I (author of this paper, that also happens to be author of the wording) have decided to review it thoughtfully again. This section contains list of possible problems that I was able to identify.

Note: Wording used for `not_fn` was largely based on existing wording of the `std::bind`, so most of the issues are common to these components.

3.1. Effects of the invocation

In the newest working draft [N4582](#) (C++ Working Draft, 2016-03-19). effects of invocation of the `std::not_fn` created wrapper are defined as follows (20.12.9 [func.not_fn]):

Returns: A forwarding call wrapper `g` such that the expression `g(a1, a2, ..., aN)` is equivalent to `! INVOKE(fd, a1, a2, ..., aN)` (20.12.2).

Where:

- `FD` is the type `decay_t<F>`,
- `fd` is an lvalue of type `FD` constructed from `std::forward<F>(f)`,
- `g` is a forwarding call wrapper created as a result of `not_fn(f)`,

Firstly, we may notice that the definition of the `fd` explicitly states that it is **lvalue** reference, which results in the problems described in motivation section.

Secondly, above wording does not mention how `cv` qualification will affect underlining functor. To be explicit the wording only describes effects of the invocation on the temporary created as the result of `not_fn` invocation. As consequence implementation that will provide only single overload of call operator in form:

```
template<typename... Args>
decltype(auto) operator()(Args&&... args) &&
{ return std::invoke(fd, std::forward<Args>(args)...); }
```

Could be considered as standard conforming, despite is questionable usability.

The author believes that the wording should guarantee that for the every invocation `static_cast<G cv ref>(g)(args...)` is equivalent to `static_cast<FD cv ref>(fd)(args...)`, where:

- `G` is type of `g`
- `cv` is either `const` or empty
- `ref` is either `&` or `&&`

In addition current wording does not clarify if the invocation `std::not_fn(f)(args...)` are equivalent to `!std::invoke(f, args...)` in unevaluated context. Such guarantee is important for components that conditionally exposes features, depending on the validity of the call expression. Most notable example of such functionality is `std::function<R(Args...)>` that is only constructible from type `F`, that is lvalue callable with `Args...` and has return compatible with `R`.

3.2. Move/copy operations on the wrapper

Existing wording describe requirements on transferability of the created wrapper as follows (20.12.9 [func.not_fn]):

The return type shall satisfy the requirements of `MoveConstructible`. If `FD` satisfies the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note*: This implies that `FD` is `MoveConstructible`. — *end note*]

The wording is requiring that the wrapper will expose the same set of the operations as the wrapped callable, however it is not defined how these operation would be defined in terms of corresponding operations on the nested functor type `FD`. As example in case when the `FD` is `CopyConstructible`, conforming implementation of the wrapper could expose only copy constructor (without declaration of move constructor) as every `CopyConstructible` is by definition `MoveConstructible`.

In addition is is not specified if the exposed move operation of the wrapper would have the same exception specification as corresponding operation of the underlining wrapper. As consequence wrapping a function pointer or simple lambda object into `not_fn`, may disable optimizations that are requiring nothrowing move operations, like using small object optimization in `std::function`.

4. Minimal wording

This section presents a minimalistic wording change that will address only lack of the value category propagation for the `not_fn`, which in author opinion is most important issue amongst all discussed in this paper.

The proposed wording changes refer to [N4582](#) (C++ Working Draft, 2016-03-19).

Apply following changes to section 20.12.9 Function template `not_fn` [func.not_fn] to:

```
template <class F>
    unspecified not_fn(F&& f);
```

In the text that follows:

- `FD` is the type `decay_t<F>`,
- `fd` is an lvalue of type `FD` constructed from `std::forward<F>(f)`,
- `g` is a forwarding call wrapper created as a result of `not_fn(f)`,

Requires: `is_constructible<FD, F>::value` shall be `true`. `fd` shall be a callable object ([func.def] 20.9.1).

Returns: A forwarding call wrapper `g` such that the expression `g(a1, a2, ..., aN)` is equivalent to `!INVOKE(static_cast<FD cv ref>(fd), a1, a2, ..., aN)` ([func.require] 20.9.2), where `cv` represents `cv-`

qualifiers of `g` and `ref` is `&` when `g` is *lvalue* and `&&` otherwise. The *cv* shall be neither *volatile* nor *const volatile*.

Throws: Nothing unless the construction of `fd` throws an exception.

Remarks: The return type shall satisfy the requirements of `MoveConstructible`. If `FD` satisfies the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note:* This implies that `FD` is `MoveConstructible`. — *end note*]

5. Comprehensive wording

This wording, present alternative approach that instead of trying to indirectly state requirements of created functor, describes them in terms of exposition only class. The author believes that such approach leads to cleaner specification.

In addition the wording is accompanied with authors drafting notes, that uses same formatting as this paragraph.

The proposed wording changes refer to [N4582](#) (C++ Working Draft, 2016-03-19).

Change the section 20.12.9 Function template `not_fn` [`func.not_fn`] to:

```
template <class F>
    unspecified not_fn(F&& f);
```

Effects: Equivalent to: `return call_wrapper(std::forward<F>(f))`, where `call_wrapper` is exposition only class defined as follows:

Use of "Effects: Equivalent to" allow us to inherit specification from the `call_wrapper` constructor.

```
class call_wrapper
{
```

Exposition only `call_wrapper` class is defined for specific invocation of `not_fn` with template parameter `F`, so it does not need to be template.

```
    using FD = decay_t<F>;
public:
    explicit call_wrapper(F&& f) noexcept(is_nothrow_constructible_v<FD, F>);
```

As above, this class works for specific type `F`, so perfect forwarding is not needed.

```
    call_wrapper(call_wrapper&&) = default;
    call_wrapper(call_wrapper const&) = default;
```

Defaulting special member functions on first declaration guarantee that they will have same affects and exception specification as the default generated ones.

```
template<typename... Args>
    auto operator()(Args&&...) & -> decltype(!declval<result_of_t<FD&(Args...)>>());

template<typename... Args>
    auto operator()(Args&&...) const& -> decltype(!declval<result_of_t<FD const&(Args...)>>());

template<typename... Args>
    auto operator()(Args&&...) && -> decltype(!declval<result_of_t<FD(Args...)>>());

template<typename... Args>
    auto operator()(Args&&...) const&& -> decltype(!declval<result_of_t<FD const(Args...)>>());
```

Listing call operators clearly indicates that both `const` and reference qualification are supported. `volatile` overloads are intentionally omitted to follow resolution of [LWG issue #2487](#).

```
private:
    FD fd;
```

```
};
```

```
explicit call_wrapper(F&& f) noexcept(is_nothrow_constructible_v<FD, F>);
```

Requires: FD shall satisfy the requirements of MoveConstructible. is_constructible_v<FD, F> shall be true. fd shall be a callable object ([func.def] 20.12.1).

Effects: Initializes fd from std::forward<F>(f).

Throws: Any exception thrown by construction of fd.

```
template<typename... Args> auto operator()(Args&&...) & -> decltype(!declval<result_of_t<FD&(Args...)>>());  
template<typename... Args> auto operator()(Args&&...) const& -> decltype(!declval<result_of_t<FD const&  
(Args...)>>());
```

Effects: Equivalent to: return !*INVOKE*(fd, std::forward<Args>(args)...) ([func.require] 20.12.2).

Use of "Effects: Equivalent to" requires that effects of invocation are the same as corresponding *INVOKE* expression even in SFINAE context. In addition use of result_of in specification of return type gives same effects.

```
template<typename... Args> auto operator()(Args&&...) && -> decltype(!declval<result_of_t<FD(Args...)>>());  
template<typename... Args> auto operator()(Args&&...) const&& -> decltype(!declval<result_of_t<FD  
const(Args...)>>());
```

Effects: Equivalent to: return !*INVOKE*(std::move(fd), std::forward<Args>(args)...) ([func.require] 20.12.2).

For the rvalue qualified call operators, invocation is performed on rvalue reference to FD type.

6. Acknowledgements

Stephan T. Lavavej suggested numerous corrections to [comprehensive wording](#) presented in the paper.

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal, and for sponsoring author's trip to the Oulu for WG21 meeting.

7. References

1. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4582, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4582.pdf>)